

System Analysis & Design

Study Guide with Answer Key: Week 1 – Week 8

Concept Overviews • Real-Life Examples • Practice Questions with Model Answers (Theory & Scenario-Based)

Course: CSE0613-2201 System Analysis & Design
University of Global Village (UGV)

WEEK 1 & 2 — Systems Analysis & Design Basics

1. Systems Analysis vs. Systems Design

Systems development is a systematic process that moves through planning, analysis, design, deployment, and maintenance. Within this process, **System Analysis** is the act of collecting and interpreting facts, diagnosing problems, and breaking a system into its components to understand its objectives — in short, it defines *what* the system should do. **System Design** then takes over: it plans a new system (or reworks an old one) by defining the components/modules needed to satisfy the requirements identified in analysis — it focuses on *how* the objective will be accomplished.

Real-Life Example: A university wants to replace its manual, paper-based admission process. During **analysis**, a team interviews admissions staff and students to find out what's slow (long queues, lost forms) and what the new system must achieve (online applications, automatic seat allocation). During **design**, the team then decides on the database structure, the web application screens, and the servers needed to make that happen.

A. Technical / Theory Questions

Q1. Define System Analysis and System Design. and explain how they differ in purpose.

Answer: System Analysis is the process of collecting and interpreting facts, identifying problems, and decomposing a system to understand its objectives — it defines *what* the system should do. System Design is the process of planning a new or replacement system by defining its components to satisfy the requirements — it defines *how* the objective will be accomplished. Analysis is problem-focused; design is solution-focused.

Q2. List the five phases of systems development in order.

Answer: Planning, Analysis, Design, Deployment (Implementation), and Maintenance.

Q3. "Analysis specifies what. design specifies how." Explain this statement with justification.

Answer: During analysis, the team determines the objectives and requirements a system must satisfy, without committing to a technical solution (the "what"). During design, the team decides the actual architecture, modules, and technology needed to meet those requirements (the "how"). This ordering ensures the solution is built to solve the right problem rather than an assumed one.

Q4. Why must system analysis be completed before system design begins?

Answer: Design decisions (database structure, interfaces, architecture) must satisfy the requirements identified during analysis. Without a completed analysis, designers would be guessing at requirements, risking a system that is technically sound but solves the wrong problem, leading to costly rework.

B. Scenario-Based Questions

Q1. A retail company's manager says, "Let's just start coding the new inventory app right away." As a system analyst, explain what step is being skipped and why it is risky.

Answer: The analysis phase is being skipped. Without gathering and documenting requirements first (current pain points, stock-tracking needs, user roles), the developed app may miss critical business needs, leading to expensive redesign, wasted development time, and a product that does not solve the actual inventory problems.

Q2. You are assigned to replace a hospital's paper patient-record system. Outline what activities would fall under analysis and what would fall under design.

Answer: Analysis: interviewing doctors/nurses/administrators, observing current record-keeping workflows, reviewing existing paper forms, and documenting requirements (e.g., "records must be searchable by patient ID"). Design: creating the database schema for patient records, designing the user interface for staff, defining system security/access controls, and specifying how the system integrates with existing hospital equipment.

2. Properties of Systems

Every system, whether organizational or technical, shares five core properties: **Organization** (structured arrangement of components to achieve objectives), **Interaction** (the way components operate with one another), **Interdependence** (components rely on each other's output as input), **Integration** (parts work together as a unified whole even while performing unique functions), and **Central Objective** (a single, clearly understood purpose that guides the whole system).

Real-Life Example: In a bank, the *loans department*, *accounts department*, and *customer service desk* are organized into a structure (Organization). The loans department interacts with the accounts department to disburse funds (Interaction). Accounts depends on data from customer service to verify identity (Interdependence). All departments work together toward one shared goal — serving the customer reliably (Integration and Central Objective).

A. Technical / Theory Questions

Q1. List and briefly explain the five properties of systems.

Answer: Organization — structured arrangement of components to meet objectives. Interaction — how components operate with each other. Interdependence — components rely on each other's output as input. Integration — parts function together as a whole despite unique roles. Central Objective — one clear, shared purpose guiding the system.

Q2. Differentiate between Interaction and Interdependence with your own definitions.

Answer: Interaction describes the manner in which components communicate or operate with one another (e.g., two departments coordinating). Interdependence goes further: it means one component's output is required as another component's input, so proper functioning of one depends directly on the other.

Q3. Why must the central objective of a system be clearly known to users before design begins?

Answer: If users and designers disagree on (or misunderstand) the system's core purpose, the resulting design may optimize for the wrong outcome. A clear, shared objective ensures the design and conversion process is aligned and successful.

B. Scenario-Based Questions

Q1. In an e-commerce company, the warehouse team ships orders only after the payment team confirms payment. Which property of systems does this best illustrate? Justify your answer.

Answer: This illustrates Interdependence: the warehouse (one subsystem) requires the output of the payment team (another subsystem) — the payment confirmation — as its own required input before it can act.

Q2. A startup's departments frequently work in isolation and rarely share information, causing conflicting goals. Which system property is missing, and what problems could this cause?

Answer: Integration (and to some extent Central Objective) is missing. Without integration, departments function as disconnected silos rather than a unified whole, which can cause duplicated effort, conflicting priorities, and failure to achieve the organization's overall goal.

WEEK 3 & 4 — Elements, Types of Systems & Categories of Information

3. Elements of a System

A system is built from six elements. **Inputs/Outputs**: data entering for processing and the useful result produced. **Processor(s)**: the component that transforms input into output. **Control**: the decision-making subsystem that governs input, processing, and output, guided by **feedback** (positive feedback reinforces good performance; negative feedback signals correction). **Environment**: the external "super-system" (e.g., vendors, competitors, regulations) that influences how the system must function. **Boundaries and Interface**: the limits that separate the system from other systems and define how it connects to them.

Real-Life Example: In an ATM system: a customer's card and PIN are *inputs*; the cash dispensed is the *output*; the ATM's internal software is the *processor*; the bank's fraud-detection rules act as *control* (declining a withdrawal that exceeds the daily limit is negative feedback); banking regulations and network uptime form the *environment*; and the ATM's card slot/screen/network connection mark its *boundary and interface* with the wider banking system.

A. Technical / Theory Questions

Q1. Name the six elements of a system and give one-line definition for each.

Answer: Inputs (data entering for processing), Outputs (the useful result produced), Processor (transforms input into output), Control (decision-making subsystem governing the system, guided by feedback), Environment (external super-system influencing the system), and Boundaries/Interface (limits defining the system and its connections to other systems).

Q2. Differentiate between positive and negative feedback with an example of each.

Answer: Positive feedback is routine and encourages/reinforces current performance (e.g., a sales system continuing a promotion because it is meeting its target). Negative feedback is corrective — it signals that something is off and prompts action (e.g., an ATM declining a withdrawal that exceeds the daily limit, prompting the control system to stop the transaction).

Q3. Explain why a system's boundary is important when designing its interface with other systems.

Answer: The boundary defines exactly what is inside vs. outside the system's scope. Knowing this boundary is crucial for designing a correct interface, since the interface is precisely the connection point where the system exchanges data or control with other systems.

B. Scenario-Based Questions

Q1. In an online food-delivery app, identify the input, processor, output, and control elements.

Answer: Input: the customer's order details and payment information. Processor: the app's backend logic that matches orders to restaurants and riders. Output: the confirmed order and delivery to the customer. Control: the system logic that rejects orders outside the delivery radius or flags suspicious payments, using feedback to keep operations within acceptable limits.

Q2. A factory's quality-control unit rejects products that don't meet standards and sends a report back to production. Identify which system element(s) this describes and explain your reasoning.

Answer: This describes the Control element, specifically operating through negative feedback: the quality-control unit makes a decision (reject the product) and sends corrective information back to the production process so it can adjust and stay within specification.

4. Types of Systems

Systems can be classified along several dimensions: **Physical or Abstract** (tangible machines vs. conceptual models/formulas), **Open or Closed** (interacts with its environment vs. isolated from it), **Adaptive or**

Non-Adaptive (responds to environmental change vs. does not), **Permanent or Temporary** (long-lasting vs. built for a fixed purpose/time), **Natural or Manufactured** (created by nature vs. man-made), and **Man-Made Information Systems** (interconnected resources — hardware, software, data — managed under direct organizational control).

Real-Life Example: A weather forecasting model is an *abstract, open, adaptive* system — it is a mathematical representation (abstract) that continuously takes in new atmospheric data (open) and adjusts its predictions as conditions change (adaptive). A stage setup for a one-night concert is a *temporary, manufactured, physical* system — built for a specific event and dismantled afterward.

A. Technical / Theory Questions

Q1. List the six ways systems can be classified, with one example of each.

Answer: Physical (a computer) or Abstract (a mathematical formula); Open (a business reacting to market changes) or Closed (a sealed laboratory experiment); Adaptive (a human being) or Non-Adaptive (a simple machine); Permanent (a country's constitution/policies) or Temporary (a concert's sound system); Natural (the solar system) or Manufactured (a train); and Man-Made Information Systems (a company's payroll system).

Q2. Explain the difference between an open system and a closed system, and state why a truly closed system is rare in reality.

Answer: An open system interacts with its environment, receiving inputs from and sending outputs to the outside world (e.g., an information system adapting to changing conditions). A closed system is isolated from environmental influence. Truly closed systems are rare because almost every real system is affected by some external factor, even if only minimally.

Q3. Give an example each of a natural system and a manufactured system.

Answer: Natural system: the solar system (or a seasonal weather system). Manufactured system: a rocket, dam, or train — created by humans.

B. Scenario-Based Questions

Q1. A company sets up a temporary help-desk booth for a three-day trade fair. Classify this system using at least three of the classification types discussed, with justification.

Answer: It is Temporary (built for the three-day event and dismantled afterward), Physical (a tangible booth with staff and equipment), and Manufactured (man-made, not naturally occurring). It could also be considered Open, since it interacts with fairgoers and adapts to visitor questions.

Q2. A smart thermostat adjusts room temperature automatically based on outside weather. Identify which system types apply to it and explain why.

Answer: It is an Open system (it takes in environmental data such as outside temperature), Adaptive (it changes its behavior in response to that data to maintain comfort), and a Man-Made Information System /Physical system (it is a manufactured device using sensors, software, and hardware).

5. Categories of Information

Organizational information is grouped by the management level that uses it. **Strategic Information** supports top management's long-range planning (e.g., revenue trends, investment decisions) and is typically produced by a **Decision Support System (DSS)**. **Managerial Information** supports middle management's short/intermediate planning (e.g., sales analysis, cash-flow projections), typically via a **Management Information System (MIS)**. **Operational Information** supports day-to-day activities of lower management (e.g., attendance, stock levels), typically via a **Data Processing System (DPS)**.

Real-Life Example: In a supermarket chain: the *CEO* uses a DSS report on five-year expansion trends (Strategic); the *regional sales manager* uses an MIS report comparing monthly sales across branches (Managerial); and the *store supervisor* uses a DPS report showing today's cash register totals and stock-outs (Operational).

A. Technical / Theory Questions

Q1. Define Strategic, Managerial, and Operational information, and name the system typically used to generate each.

Answer: Strategic Information supports top management's long-range planning (generated via a Decision Support System, DSS). Managerial Information supports middle management's short/intermediate-term planning (generated via a Management Information System, MIS). Operational Information supports day-to-day activities for lower management (generated via a Data Processing System, DPS).

Q2. Match each information category to the management level that primarily uses it.

Answer: Strategic → Top management. Managerial → Middle management. Operational → Lower (operational-level) management.

Q3. Why would a top executive rarely need daily attendance records?

Answer: Daily attendance records are operational-level detail relevant to day-to-day supervision. Top executives focus on long-range, high-level planning (e.g., growth strategy), so granular daily data is not directly useful at their decision-making level — it would typically be summarized before reaching them, if at all.

B. Scenario-Based Questions

Q1. A logistics company's fleet manager wants a report on "which trucks are idle right now" while the CEO wants a report on "whether to expand into a new country over the next five years." Classify each request into its correct information category.

Answer: The fleet manager's request ("which trucks are idle right now") is Operational Information, supporting immediate day-to-day decisions. The CEO's request (five-year expansion decision) is Strategic Information, supporting long-range planning.

Q2. Explain, with a real business example, how the same raw data (e.g., sales figures) could be processed differently to serve strategic, managerial, and operational needs.

Answer: Raw daily sales transactions can be used operationally as-is (e.g., today's register totals for a store supervisor). The same data aggregated monthly by branch becomes managerial information (e.g., a regional manager comparing branch performance). Aggregated over several years and combined with market trends, it becomes strategic information (e.g., a CEO deciding whether to expand).

WEEK 5 & 6 — Analysis vs. Design, Role & Qualities of the Analyst

6. Differences Between System Analysis and System Design

Though used interchangeably, analysis and design are distinct phases with different focus, output, techniques, and level of detail. Analysis focuses on **understanding the problem** and gathering requirements (output: a requirements document; techniques: interviews, surveys, observation, document analysis; high-level understanding). Design focuses on **specifying the solution** (output: design specifications; techniques: UML, DFDs, ERDs, decision trees, state transition diagrams; detailed specifications).

Real-Life Example: When a school wants a new library management system, the analysis phase produces a document stating "students must be able to search, reserve, and renew books online." The design phase then produces an ER diagram of the book/student database and UML diagrams showing exactly how the reservation workflow will function in the software.

A. Technical / Theory Questions

Q1. Tabulate at least three differences between system analysis and system design.

Answer: (1) Focus: Analysis understands the problem domain and gathers requirements; Design specifies the solution. (2) Output: Analysis produces a requirements document; Design produces system design specifications. (3) Techniques: Analysis uses interviews, surveys, observation, and document analysis; Design uses UML, DFDs, ERDs, decision trees, and state transition diagrams. (4) Level of detail: Analysis gives a high-level understanding; Design gives detailed specifications.

Q2. Why is a requirements document considered the main output of the analysis phase?

Answer: The requirements document captures everything discovered about what the system must do — the needs, constraints, and objectives of stakeholders. It becomes the reference point that the design phase must satisfy, making it the key deliverable of analysis.

Q3. Name two techniques each that are typically used in analysis and in design.

Answer: Analysis: Interviews and Surveys (also observation or document analysis). Design: UML and Data Flow Diagrams (also ERDs, decision trees, or state transition diagrams).

B. Scenario-Based Questions

Q1. A project team skips writing formal requirements and jumps directly into drawing ER diagrams. What risks does this create for the final product? Relate your answer to the differences between analysis and design.

Answer: Since design specifications should be derived from and validated against requirements, skipping analysis means the ER diagrams may reflect the team's assumptions rather than actual stakeholder needs. This raises the risk of building a technically well-designed system that fails to satisfy real business requirements, leading to costly rework later.

Q2. You are reviewing a project document that only lists database schema diagrams and UML class diagrams. Which phase does this document most likely belong to, and what might be missing from the project?

Answer: This document belongs to the Design phase, since schema diagrams and UML class diagrams are design techniques/outputs. What's likely missing is the Analysis-phase requirements document that should have preceded and justified these design decisions.

7. Role of the System Analyst & 8. Qualities of a Good Analyst

The **system analyst** plays a central role in an information-system project: applying technology to solve business problems, and often acting as a **change agent** who identifies opportunities for organizational improvement. To do this effectively, a good analyst needs to be a **problem solver**, a strong **communicator** (translating between technical staff and non-technical stakeholders), possess strong **personal and professional ethics** (handling sensitive business/user data responsibly), and be **self-disciplined and self-motivated** (managing long, ambiguous projects without constant supervision).

Real-Life Example: When a manufacturing firm wants to digitize its supply chain, the analyst interviews floor workers, warehouse staff, and executives (communicator), proposes a barcode-scanning solution to a paper-based bottleneck (problem solver and change agent), keeps confidential supplier pricing data secure (ethics), and continues researching best practices even without being asked (self-motivation).

A. Technical / Theory Questions

Q1. What is meant by describing a system analyst as a "change agent"?

Answer: It means the analyst does more than just build a technical solution — they actively identify opportunities to improve how the organization operates and help drive that organizational improvement forward, often influencing processes and people beyond the software itself.

Q2. List the four key qualities of a good system analyst.

Answer: Problem solver, strong communicator, strong personal and professional ethics, and self-disciplined and self-motivated.

Q3. Why is communication considered one of the most critical skills for a system analyst?

Answer: The analyst constantly bridges the gap between non-technical stakeholders (who describe needs in business terms) and technical teams (who need precise specifications). Without strong communication, requirements get lost or misinterpreted, leading to a system that doesn't meet real needs.

B. Scenario-Based Questions

Q1. During requirement-gathering, an analyst discovers that adopting the client's preferred solution would actually harm business efficiency. What should the analyst do, and which of the analyst's qualities does this test?

Answer: The analyst should honestly explain the concern to the client, backed with evidence, and propose a better alternative rather than silently complying. This tests the analyst's ethics (honesty over convenience) and problem-solving ability (finding and justifying a better solution).

Q2. A newly hired analyst is given a vague, ambiguous problem statement with little supervision. Which qualities will most help them succeed, and how would they apply each one in this situation?

Answer: Self-discipline and self-motivation will help them proactively research and clarify the problem without waiting for direction. Problem-solving skills help them break the ambiguous statement into manageable questions, and communication skills help them consult stakeholders to fill in the gaps.

WEEK 7 & 8 — Techniques, Design Activities & Scaling

9. Techniques Used in System Analysis

Analysts gather requirements using several fact-finding techniques: **Interviews** (face-to-face or online conversations with stakeholders), **Surveys** (questionnaires to reach many respondents efficiently), **Observation** (watching the current system in operation), **Document Analysis** (examining existing reports, forms, and manuals), and **Prototyping** (building simplified mock-ups to gather feedback and refine requirements).

Real-Life Example: When redesigning a hospital's patient check-in process, analysts might *interview* receptionists about pain points, send a *survey* to patients about wait-time satisfaction, *observe* the check-in desk during a busy morning, *analyze* the existing paper intake forms, and finally build a clickable *prototype* of a new tablet-based check-in screen for staff to react to before full development begins.

A. Technical / Theory Questions

Q1. List the five fact-finding techniques used in system analysis.

Answer: Interviews, Surveys, Observation, Document Analysis, and Prototyping.

Q2. Explain the difference between an interview and a survey as data-gathering techniques.

Answer: An interview is a direct, face-to-face or online conversation with a stakeholder, allowing in-depth, flexible follow-up questions but reaching only one person at a time. A survey uses standardized questionnaires distributed to many respondents at once, trading depth for breadth and efficiency.

Q3. What is the main purpose of prototyping during the analysis phase?

Answer: Prototyping creates a simplified model or mock-up of the proposed system so stakeholders can interact with something concrete, give feedback early, and help refine and validate requirements before full-scale development begins.

B. Scenario-Based Questions

Q1. You need to gather requirements from 500 geographically spread customers of a mobile banking app within a week. Which technique(s) would you choose and why?

Answer: A survey (questionnaire) would be the most practical choice, since it can efficiently collect data from a large, geographically dispersed group within a short timeframe, unlike interviews which would be too time-consuming for 500 people individually.

Q2. An analyst wants to understand exactly how clerks currently process loan applications, but the clerks are not very good at articulating their own workflow in interviews. Which technique should the analyst rely on instead, and why?

Answer: Observation would be more effective here. By directly watching the clerks perform their tasks, the analyst can capture the actual workflow, including steps the clerks may not think to mention or aren't able to clearly describe verbally.

10. System Design: Key Activities & Techniques

System design is the phase where detailed specifications are produced from the analysis output. Its **key activities** are: Architectural Design (overall structure), Component Design (individual parts and their interactions), Interface Design (connections between components and external systems), Data Design (database schema/structures), and Detailed Design (algorithms and data flow for each component). Common **techniques** used include UML, Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs), Decision Trees, and State Transition Diagrams.

Real-Life Example: For a ride-sharing app, *architectural design* decides on a microservices structure (separate services for booking, payment, and maps); *data design* produces an ERD linking Drivers, Riders, and Trips tables; *interface design* specifies how the mobile app talks to the payment gateway API; and a *state transition diagram* models a trip's lifecycle: Requested → Accepted → In-Progress → Completed.

A. Technical / Theory Questions

Q1. List the five key activities involved in system design.

Answer: Architectural Design, Component Design, Interface Design, Data Design, and Detailed Design.

Q2. Name three techniques used in system design and describe what each one visually represents.

Answer: UML represents system structure and behavior through standardized diagrams (e.g., class, sequence diagrams). Data Flow Diagrams (DFDs) illustrate how data moves through the system's processes. Entity-Relationship Diagrams (ERDs) represent the data entities in a database and the relationships between them.

Q3. Differentiate between an Entity-Relationship Diagram and a State Transition Diagram.

Answer: An ERD represents the data entities of a system and the relationships between them (used for database design). A State Transition Diagram represents the different states a system (or object) can be in and the transitions between those states (used to model behavior over time).

B. Scenario-Based Questions

Q1. A team is designing a food-delivery order system. Map each of the five key design activities to a concrete deliverable you would expect for this specific system.

Answer: Architectural Design: deciding on a microservices structure separating ordering, payment, and delivery-tracking services. Component Design: designing the order-matching module and its interaction with the rider-assignment module. Interface Design: specifying how the mobile app communicates with the restaurant's order system via an API. Data Design: an ERD linking Customers, Orders, Restaurants, and Riders. Detailed Design: the exact algorithm for matching the nearest available rider to an order.

Q2. You need to model how an online exam's status changes from "Not Started" to "In Progress" to "Submitted" to "Graded." Which system design technique is most appropriate, and how would you represent it?

Answer: A State Transition Diagram is most appropriate, since it is designed to show the different states an object can be in and the events that trigger transitions between them — here, states would be Not Started, In Progress, Submitted, and Graded, connected by transition arrows labeled with the triggering actions (e.g., "student starts exam", "student submits", "teacher grades").

11. Horizontal and Vertical Scaling

Scaling means adjusting resources (compute, storage, network) so a system can handle growing demand without losing performance. **Vertical scaling** (scale-up) means adding more power (CPU, RAM) to a single machine — simple and consistent, but limited by hardware ceilings, costly at the high end, and a single point of failure. **Horizontal scaling** (scale-out) means adding more machines to share the load — offering near-unlimited growth, fault tolerance, and cost efficiency at scale, at the price of higher complexity, network overhead, and data-consistency challenges. The right choice depends on architecture (monolithic vs. distributed), budget, consistency requirements, and expected growth.

Real-Life Example: A small business upgrades its single office server with more RAM and a faster processor to keep up with growth — this is *vertical scaling*. In contrast, a video-streaming platform like a global OTT service adds hundreds of additional servers across multiple data centers during a big live-event premiere to handle a sudden surge in viewers — this is *horizontal scaling*. A delivery company that starts with one van and buys more vans as orders increase (rather than buying one giant van) is scaling horizontally too.

A. Technical / Theory Questions

Q1. Define horizontal scaling and vertical scaling, and give one advantage of each.

Answer: Vertical scaling adds more resources (CPU, RAM, storage) to a single existing machine; its advantage is simplicity, since managing one machine is less complex. Horizontal scaling adds more machines to share the workload; its advantage is near-unlimited scalability, since you can keep adding nodes as demand grows.

Q2. List two limitations of vertical scaling and two limitations of horizontal scaling.

Answer: Vertical scaling limitations: (1) it is a single point of failure — if that server fails, the whole system goes down; (2) there is a hardware ceiling on how much CPU/RAM/storage one machine can hold. Horizontal scaling limitations: (1) increased complexity in managing multiple servers; (2) data-consistency challenges across distributed nodes.

Q3. What factors should be considered when deciding between horizontal and vertical scaling for a new system?

Answer: Application architecture (monolithic vs. distributed/microservices), budget (cloud cost efficiency vs. expensive high-end hardware), consistency requirements (strict consistency favors vertical), and expected growth rate (rapid growth favors horizontal's near-unlimited scalability).

B. Scenario-Based Questions

Q1. A banking application requires strict data consistency and currently runs as a monolithic system. Would you recommend horizontal or vertical scaling as it grows? Justify your answer using the factors discussed in class.

Answer: Vertical scaling would generally be recommended initially, since monolithic applications are easier to scale vertically, and banking systems requiring strict data consistency favor vertical scaling's simpler data management on a single machine. Horizontal scaling could later be considered for specific components, but would require careful handling of the distributed consistency issues it introduces.

Q2. An e-commerce startup expects unpredictable, rapid traffic spikes during flash sales and is built on a microservices architecture. Recommend a scaling strategy and explain the trade-offs the company should be prepared for.

Answer: Horizontal scaling is recommended: microservices architectures are naturally suited to scaling out, and horizontal scaling offers the near-unlimited, elastic capacity needed for unpredictable traffic spikes, along with fault tolerance and cost efficiency in the cloud. The company should be prepared for increased operational complexity in managing many servers, potential data-consistency issues across nodes, and added network overhead from inter-node communication.